



# Testbenches & Sequential Logic

Made By Ahmed Amr & Aya Radwan  
2024/2025



# Agenda

- Testbenches using Tasks & functions
- Do file
- Intro to sequential logic
- Key differences between Sequential & Combinational
- Basic storage elements, like:- latches, Flipflops
- Blocking vs non-blocking assignment
- Synchronous Vs Asynchronous.
- Event Queue
- Fatal Design mistakes, like:- unintentional latches, race conditions

# Quick Revision

## ☐ Coding styles.

- Gate-level
- Structural
- Dataflow level
- Behavioral

## ☐ module instantiation.

- `module_name instance_name (.port_name1(signal1), .port_name2(signal2), ... );`
- Or
- `module_name instance_name (signal1, signal2, ... );`

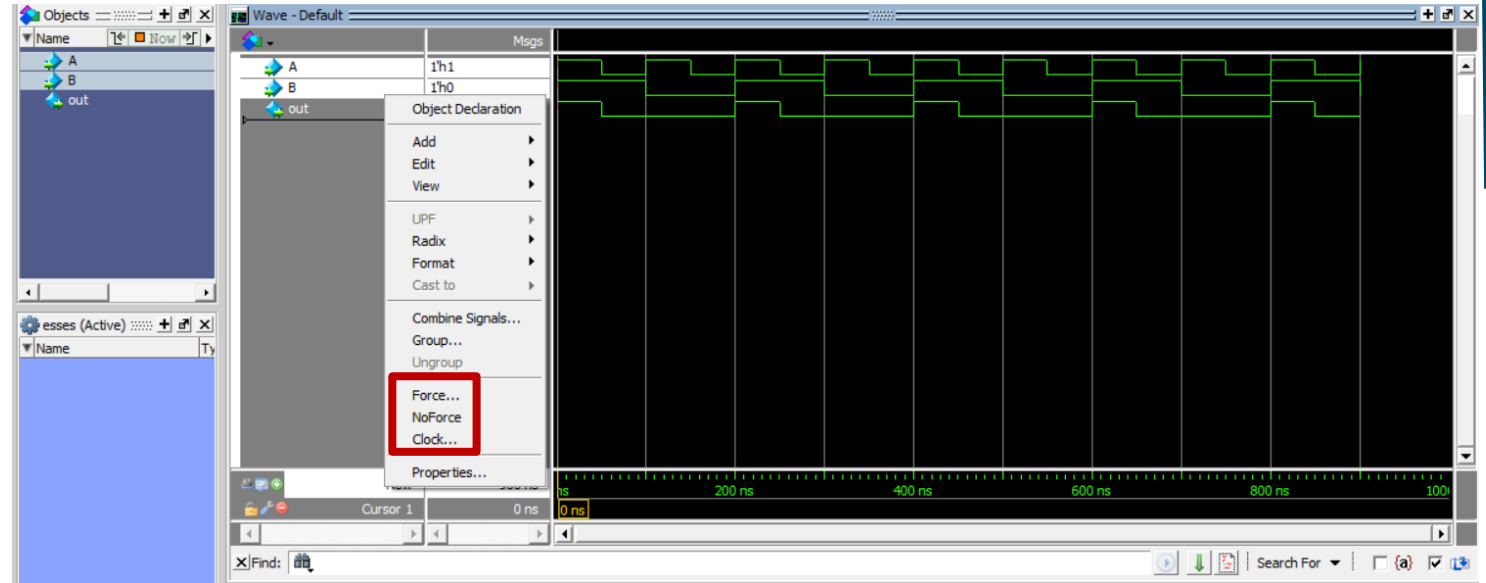
# Testbenchs

# How we used to test

Given a simple And-gate design:

```
module And_gate
(
    input wire A,
    input wire B,
    output wire out
);
    assign out = A & B;
endmodule
```

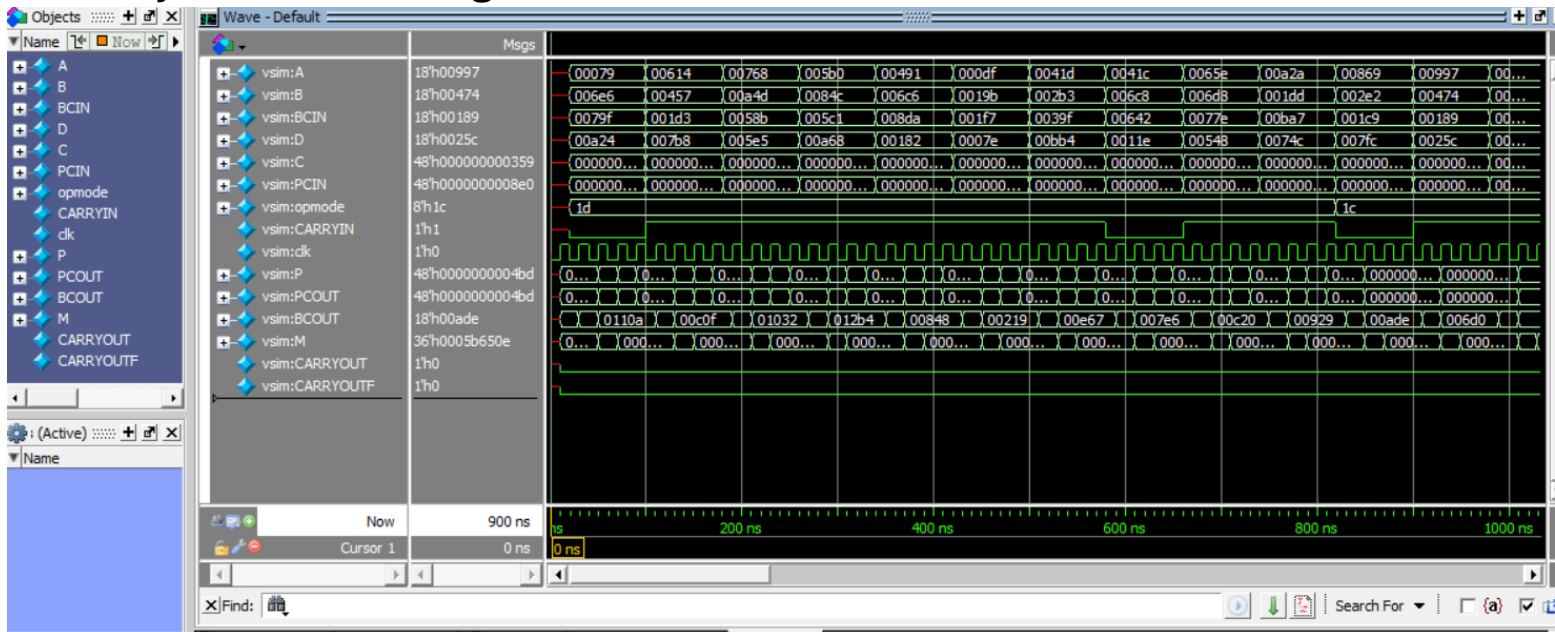
**Simulate**



Validate the design with your eyes !!

# How we used to test

Now if you have a design like this:

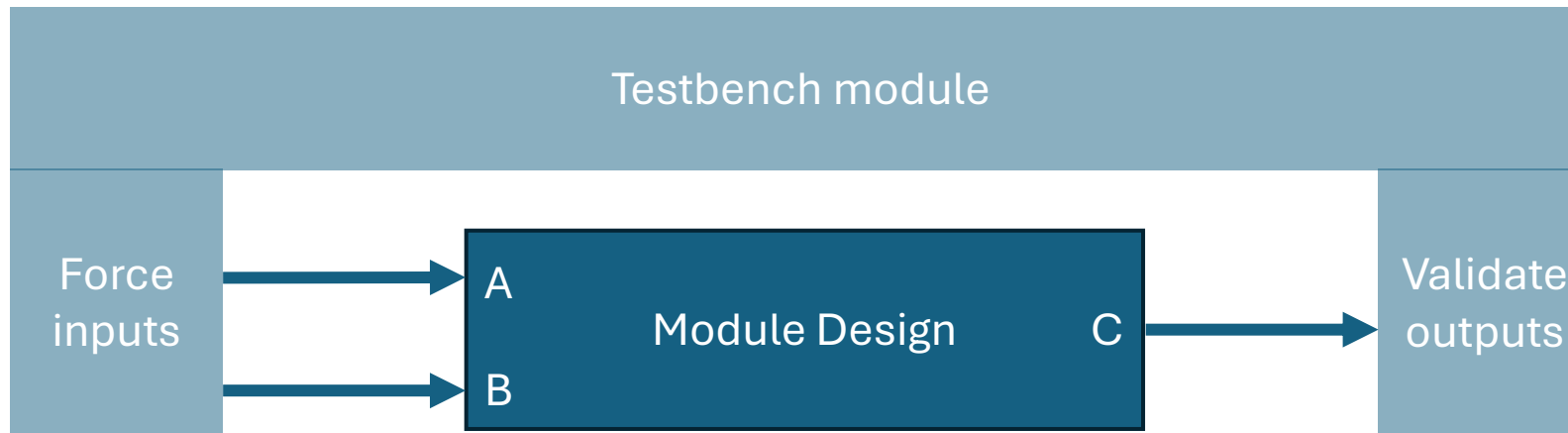


It is impossible for you to validate or even force each and every single signal !!

# What is a testbench

- ❑ This leads us to the concept of **testbench**.
- ❑ A testbench is like a **script** for the design written in Verilog.
- ❑ When you hit **run** in the simulation, this script starts **automatically**.
- ❑ It **forces the inputs** that you want and even **validates the output**.

This could be visualized as:



the testbench is not translated into a hardware. (not synthesizable)

# Testbench Structure



# Initial Block

## Syntax:

```
initial
begin
  A=0;
  B=0;
  #10;
  A=1;
  B=1;
end
```

**Hint:** begin and end are only needed when more than one statement is used inside the initial block

**Note:** all variables assigned inside initial block are of type **reg**

- ☐ Initial blocks are like always blocks, the statements are executed in sequence (procedurally)
- ☐ The difference between them is that the initial block is executed **once** at **the beginning of the stimulus**, and no sensitivity list is needed
- ☐ The main use for initial blocks, is to describe the stimulus, through initializing the variables and generating the input stream, and checking for the output signals.
- ☐ It is the building block for the testbench !!
- ☐ Initial blocks are **not synthesizable**
- ☐ **#** is used for delay (suspend the execution) with given time
- ☐ **`timescale** is used to specify the **time units** and **time precision**

# For loop & repeat block

Syntax:

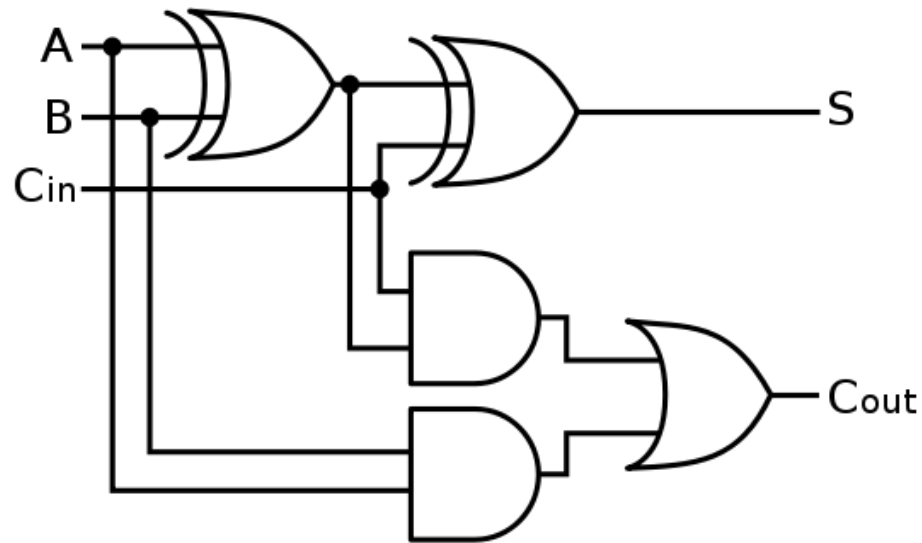
```
integer i;  
  
initial begin  
    for (i=0; i<100; i=i+1)  
    begin  
        statement;  
    end  
End
```

```
initial begin  
    repeat(100)  
    begin  
        statement;  
    end  
End
```

- ☐ Just like C/C++, for loop is used to execute a statement multiple times
- ☐ But, in Verilog, we **can't use ++ operator** to increment the I !!
- ☐ Just like the if statement and case statement, it can be **only used in procedural blocks** (always & initial blocks)
- ☐ If it contains only one statement, no need for begin ... end
- ☐ Integer i must always be declared before the for loop.
- ☐ The repeat block is like the for loop, but the counter variable is omitted.

# Example

- ❑ For this example, we will use a full-adder design.
- ❑ We will make a testbench for this circuit.



This is called **parametrized design**.  
 “**Width**” is a parameter and takes the value “4”.

```
module Full_Adder #(parameter WIDTH = 4)
(
  input [WIDTH-1:0] X1, X2,
  input Cin,
  output [WIDTH-1:0] S,
  output Cout
);

  assign {Cout, S} = X1 + X2 + Cin;

endmodule
```

# Example

- ❑ To make the testbench, create a new file, and start a new module for the testbench.

```

module full_adder_tb;

    parameter WIDTH = 4;

    reg [WIDTH-1:0] X1_tb, X2_tb;
    reg Cin_tb;

    wire [WIDTH-1:0] S_tb;
    wire Cout_tb;

    Full_Adder #(.WIDTH(4)) DUT(.X1(X1_tb),
    .X2(X2_tb), .Cin(Cin_tb), .S(S_tb),
    .Cout(Cout_tb));

    initial
    begin
        X1_tb=0;
        X2_tb=0;
        Cin_tb=0;
        #10
        $display("test case1");
        X1_tb=1;
        X2_tb=0;
        Cin_tb=1;
        #5
    end
endmodule

```

```

    if (S_tb==2 && Cout_tb==0)
        $display("test case pass");
    else
        $display("test case failed");
    #10
    $display("test case2");
    X1_tb=1;
    X2_tb=1;
    Cin_tb=1;
    #5
    if(S_tb==3 && Cout_tb==0)
        $display("test case pass");
    else
        $display("test case failed");
    #5;
    $stop;
end
endmodule

```

This is how to instantiate a parameterized design  
**“module\_name #(param value)  
 instance\_name (signal1, signal2, ... );”**

# Example

- ❑ After Writing your testbench, run it through QuestaSim.
- ❑ Watch you design come to life, all by itself!!

➤ What are those, in the code ?!

```
#10  
$display("test case1");  
X1_tb=1'b1;
```

```
if (S_tb==0 && Cout_th==1)  
    $display("test case pass");
```

```
else  
    $display("test case failed");
```

```
#5  
$stop;  
end
```

# Tasks & Functions

# Tasks & Functions

- ❑ The main difference between them is, tasks doesn't have time delay, while functions have delay
- ❑ Both of them are used in the testbench, to serve a specific role in the testbench
- ❑ System tasks and functions start with \$ sign (built-in functions):
  - \$display
    - Prints text in the transcript during simulation
    - It can also include format characters, %d (decimal), %h (hexadecimal), %b (binary), %c (character), %s (string), %t (time), %m (hierarchy level).
  - \$monitor
    - Prints the text whenever one of the signals in the signals list changes
  - \$random
    - Generate random integer ever time it is called
  - \$stop
    - Stop the simulation run
  - \$finish
    - Stop the simulation run and quits the simulation

# Tasks & Functions

- ❑ Tasks and functions can also be defined by the user.
- ❑ Syntax:

```
task taskname();  
  begin  
    statment;  
  end  
Endtask
```

```
function funcname();  
  begin  
    statment;  
  end  
Endfunction
```

- ❑ If you have repeating part in your testbench, just make a task for it !!
- ❑ The tasks and functions are placed in between **module() .... endmodule**
- ❑ To reuse the task of function in you testbench, just call it back like in C/C++



# Example

□ Lets revisit our full-adder example, with implementing a simple task.

```
module full_adder_tb;

    parameter WIDTH = 4;

    reg [WIDTH-1:0] X1_tb, X2_tb;
    reg Cin_tb;

    wire [WIDTH-1:0] S_tb;
    wire Cout_tb;

    Full_Adder #(.WIDTH(4)) DUT(.X1(X1_tb),
    .X2(X2_tb), .Cin(Cin_tb),.S(S_tb),
    .Cout(Cout_tb));

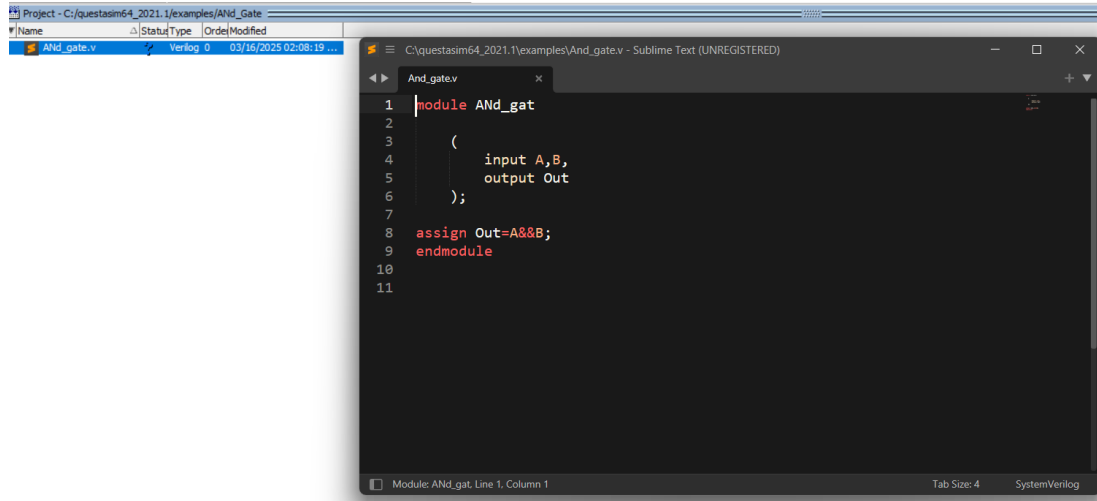
    initial
    begin
        reset();
        #10
        $display("test case1");
        X1_tb=1;
        X2_tb=0;
        Cin_tb=1;
        #5
        if (S_tb==2 && Cout_tb==0)
            $display("test case pass");
        else
            $display("test case failed")
        #10
    end
endmodule
```

```
        $display("test case2");
        X1_tb=1;
        X2_tb=1;
        Cin_tb=1;
        #5
        if(S_tb==3 && Cout_tb==0)
            $display("test case pass");
        else
            $display("test case failed");
        #5;
        $stop;
    end

    task reset();
        begin
            X1_tb=0;
            X2_tb=0;
            Cin_tb=0;
        end
    endtask
endmodule
```

# Do File

# Traditional Simulation

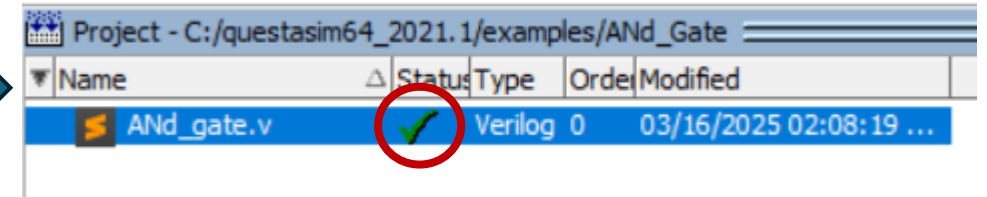


```

1 module AND_gat
2
3   (
4     input A,B,
5     output Out
6   );
7
8   assign Out=A&&B;
9 endmodule
10
11

```

Compile

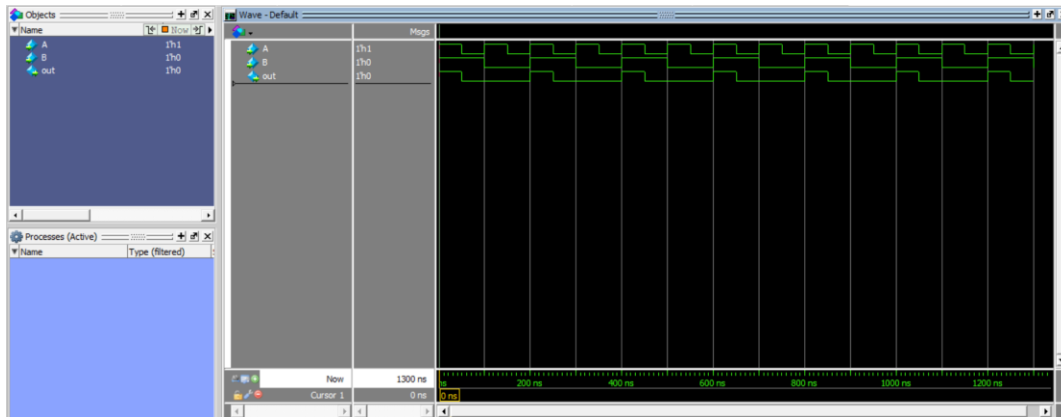



| Name       | Status | Type      | Order | Modified                |
|------------|--------|-----------|-------|-------------------------|
| ANd_gate.v | ✓      | Verilog 0 |       | 03/16/2025 02:08:19 ... |

Simulate



Run

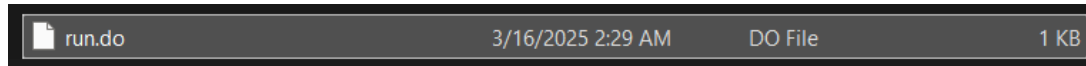


# What is a Do file

- ☐ The simulation process that was demonstrated could be **fully automated**.
- ☐ Do file is a **script** for the Questasim.
- ☐ It is a **collection of command lines** (script) for the tool.
- ☐ These command lines **run one-by-one**.
- ☐ The arrangement of these command lines are as follows:
  1. It starts by telling the tool to compile the files of our designs.
  2. Then runs the simulation.
  3. Then adds the waves to the waveform.
  4. And finally, it runs our testbench!

# How to create your Do file

- ❑ After creating your project in Questasim and writing your design & testbenches.
- ❑ Create a new file in the project folder name “**run.do**”
- ❑ Open the file you just created and start adding your command lines.
- ❑ The file runs line-by-line so make sure you have the right arrangement.



To run this file just write in the transcript “do run.do”

```
vlib work
vlog Design1.v
Design1_tb.v
vsim -voptargs=+acc
work.Module_Name
add wave *
run -all
#quit -sim
```

# Do file code

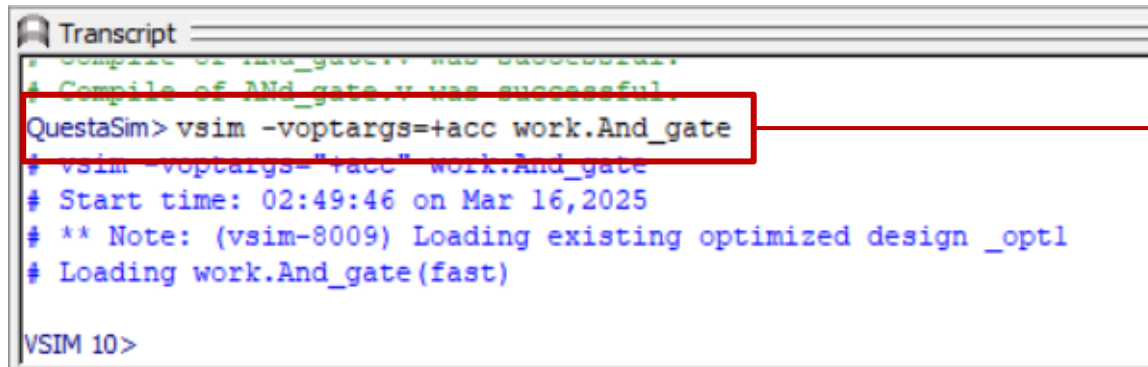
- ☐ `vlib work` → Create work library
- ☐ `vlog Design1.v Design1_tb.v` → Compile the DUT and testbench and all other files
- ☐ `vsim -voptargs=+acc work.Module_Name` → Simulates the tb using module name that was compiled in the work library
- ☐ `add wave *` → Add all waves in tb (also waves u want)
- ☐ `run -all` → Run the simulation

**Note:** if any of these command lines had an error, the tool detects that error and asks you to fix it in the transcript.

# Important Remark

- ❑ All these codes are from Questasim.
- ❑ After doing any step in Questasim, the command for this step could be seen in the transcript.

❖ Example:

A screenshot of the Questasim Transcript window. The window title is "Transcript". The text inside shows the command "QuestaSim> vsim -voptargs=+acc work.And\_gate" highlighted with a red box. Below this, it shows the execution of the command, including the start time and the loading of the design.

```
Transcript
# Compile of And_gate.v was successful.
# Compile of And_gate.v was successful.
QuestaSim> vsim -voptargs=+acc work.And_gate
# vsim -voptargs="+acc" work.And_gate
# Start time: 02:49:46 on Mar 16,2025
# ** Note: (vsim-8009) Loading existing optimized design _opt1
# Loading work.And_gate(fast)

VSIM 10>
```

After running the simulation, you can see the command in the transcript.

# Lab 1

## ALU testbench



# LAB 1

- Last session we implemented this ALU

| Op-code | Operation           |
|---------|---------------------|
| 2'b00   | $A + B$ (add)       |
| 2'b01   | $A - B$ (subtract)  |
| 2'b10   | $A \& B$ (AND gate) |
| 2'b11   | Pass A              |

| Signal | Size (bit) |
|--------|------------|
| enable | 1          |
| A      | 4          |
| B      | 4          |
| out    | 4          |

- Implement a testbench for this design
  - Testing at least 100 test cases (hint: use for loop or repeat block)
  - Randomizing the input for each testcase.

**Bonus:** create a task to Validate the output.

# Solution

```
module ALU_tb;
    // Inputs
    reg [3:0] A, B;
    reg [1:0] opcode;
    reg enable;
    // Outputs
    wire [3:0] out;
    wire cout;
    // Instantiate the ALU module
    Lab2_ALU dut (A, B, enable, opcode, out, cout);
    // Task to validate output
    task validate_output;
    begin
        case (opcode)
            2'b00: if ({cout, out} !== A + B) $display("Error: A + B failed at time %0t", $time);
            2'b01: if ({cout, out} !== A - B) $display("Error: A - B failed at time %0t", $time);
            2'b10: if (out !== (A & B)) $display("Error: A & B failed at time %0t", $time);
            2'b11: if (out !== A) $display("Error: Pass A failed at time %0t", $time);
        endcase
    end
endtask
```

# Solution

```

initial begin
    A = 0; B = 0; opcode = 0; enable = 0;
    #10// Enable ALU
    enable = 1;
    // Randomized testing for 100 test cases
    repeat (100) begin
        A = $random;
        B = $random;
        opcode = $random;
        #10; // Wait for 10 time units before the next test case
        validate_output();
    end
    // Disable ALU
    enable = 0;
    // End simulation
    #10
    $stop;
end
// Monitor changes
initial begin
    $monitor("Time=%0t | A=%b | B=%b | Opcode=%b | Enable=%b | Out=%b | Cout=%b",
    $time, A, B, opcode, enable, out, cout);
end
endmodule

```

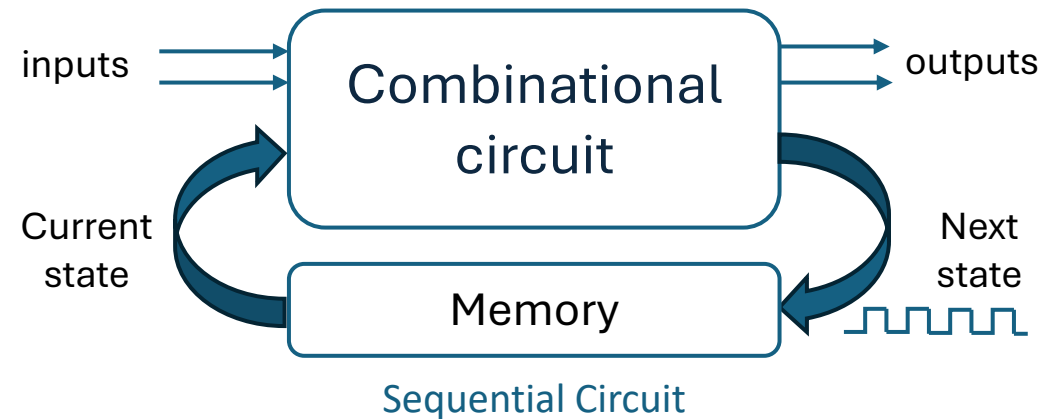


# Break

# Combinational vs. Sequential logic

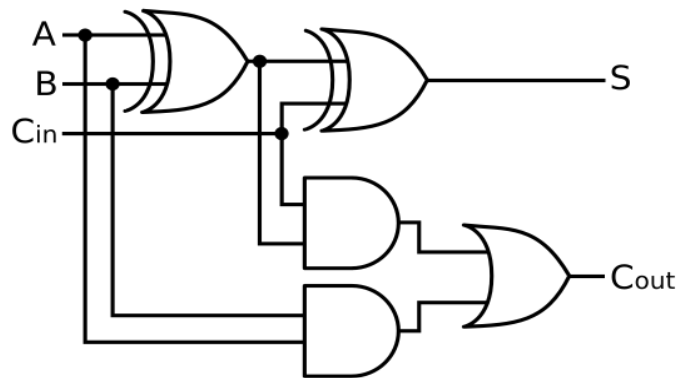
# Combinational vs. Sequential

- Combinational Circuits → Output depends only on current inputs.
- Sequential Circuits → Output depends on current & past input
- Combinational → No memory (No past input storage).
- Sequential → Has memory (Uses flip-flops or latches).

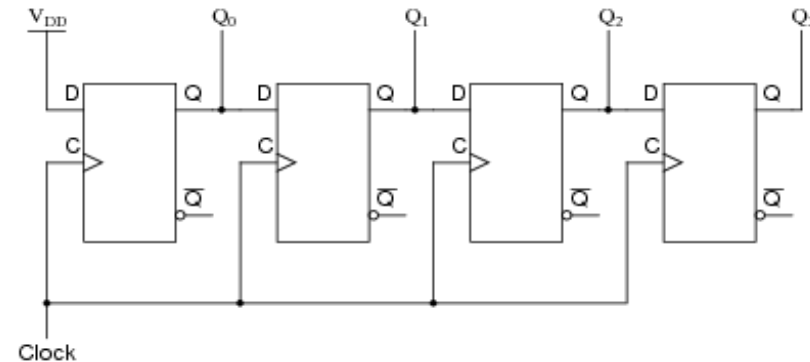


# Combinational vs. Sequential

- Combinational → Immediate response (No timing concerns).
- Sequential → Time-dependent (Output depends on timing and clock).
- Combinational → Adders, Multiplexers, Encoders.
- Sequential → Counters, Registers, Finite State Machines (FSMs).



Adder Circuit



Shift Register Circuit

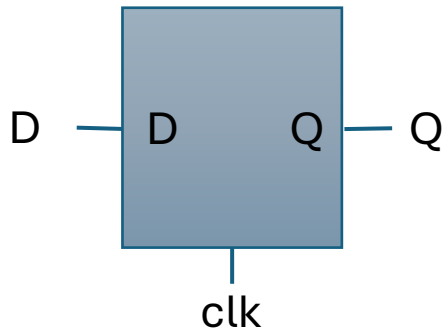
# Introduction To Storage Elements



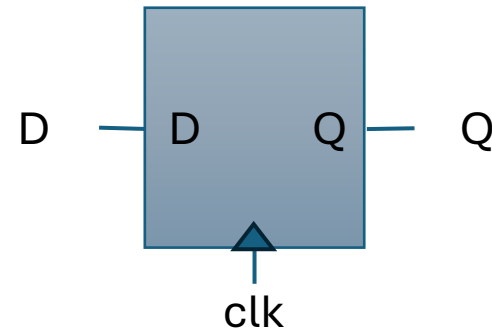
# Intro to storage elements

- Sequential logic circuits depend on both current and past input values, meaning they have memory. The circuit might explicitly remember previous inputs or distill them into a smaller set called the *state*.
- Types of Storage Elements:

a) **Latch** (Level Sensitive)



b) **Flip-flop** (Edge-triggered)



# Flip-Flops vs. Latches

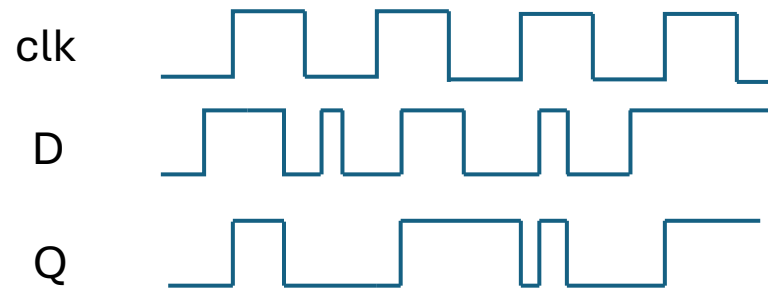
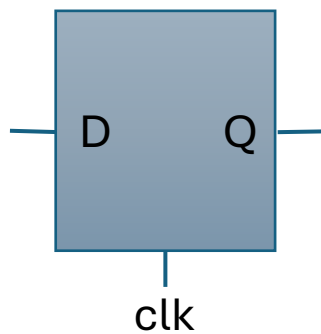
# Latches

- Basic storage elements that store data based on the level of the control signal (e.g., a clock or enable signal).
- **Level-sensitive:** The data is stored when the control signal is active (high or low).
- Latches are **rarely** used independently in standard digital design flows. They are often **unintentionally inferred** during the design process.
- Types of latches: D Latch, SR Latch, JK Latch, and T Latch.

## D Latch (Transparent Latch)

→ Positive (Level) D Latch:

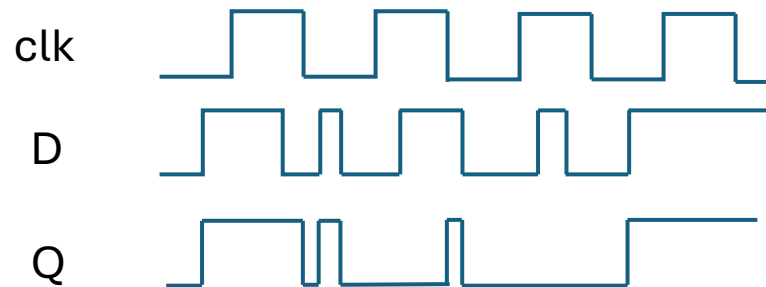
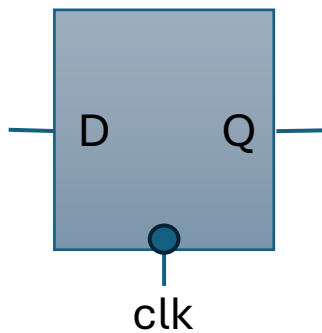
- When the clock signal is high, the output follows the input ( $Q = D$ ).
- When the clock signal goes low, the output holds its previous state, storing the input.



## D Latch (Transparent Latch)

→ Negative (Level) D Latch:

- When the clock signal is low, the output follows the input ( $Q = D$ ).
- When the clock signal goes high, the output holds its previous state, storing the input.



# Positive vs. Negative D Latch

```
module pos_dlatch (  
    input clk,  
    input d,  
    output reg q  
);  
    always @(*) begin  
        if (clk) begin  
            q <= d;  
        end  
    end  
endmodule
```

```
module neg_dlatch (  
    input clk,  
    input d,  
    output reg q  
);  
    always @(*) begin  
        if (~clk) begin  
            q <= d;  
        end  
    end  
endmodule
```

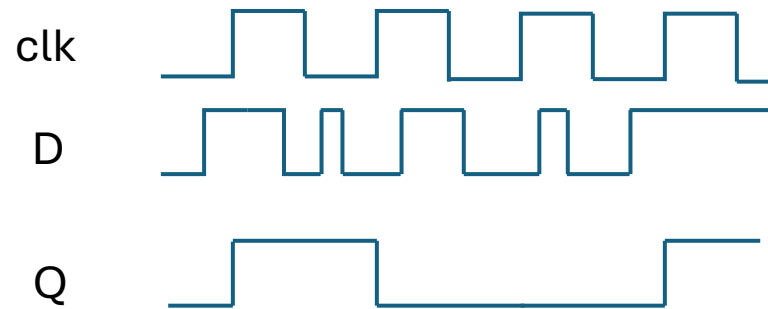
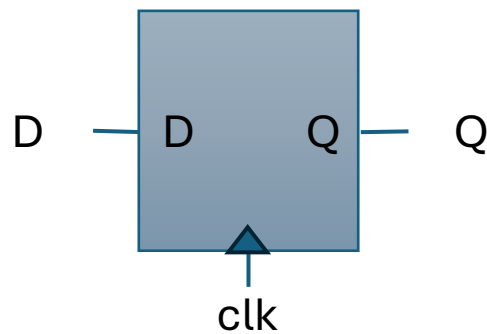
# Flip-Flops

- Basic storage elements that store data based on the edge of a control signal (e.g., a clock signal).
- **Edge-triggered**: Data is stored only on the rising or falling edge of the clock.
- **More stable and widely used** in digital design compared to latches.
- Types of Flip-flop : D Flip-flop, SR Flip-flop, JK Flip-flop, and T Flip-flop.

## D Flip-Flop

→ Rising Edge D Flip-Flop:

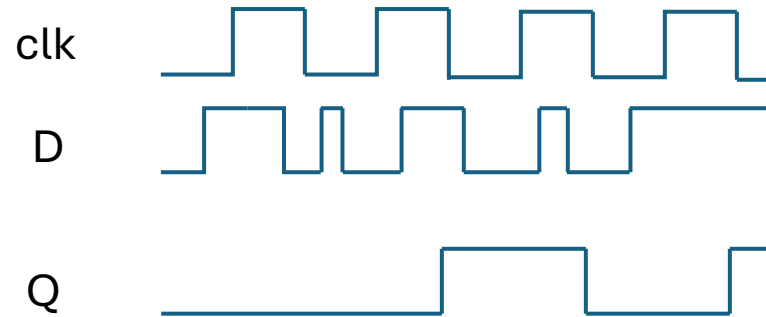
- samples the input value when the clock transitions from **low to high** (↑)
- After the rising edge, the output remains stable until the next clock edge





## → Falling Edge D Flip-Flop:

- 
- Diagram of a D flip-flop. A square block has an input 'D' on the left and an output 'Q' on the right. A clock input 'clk' is shown at the bottom with an upward-pointing triangle.



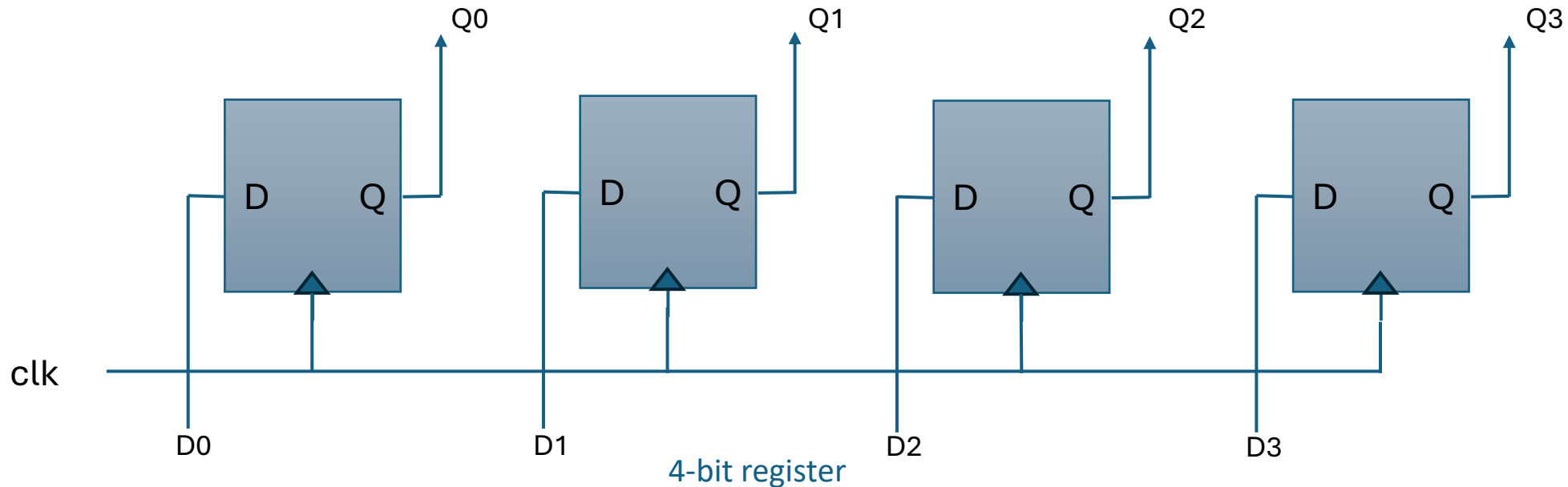
# Positive vs. Negative edge D-FF

```
module posedge_dff (  
    input clk,  
    input d,  
    output reg q  
);  
    always @(posedge clk) begin  
        q <= d;  
    end  
endmodule
```

```
module negedge_dff (  
    input clk,  
    input d,  
    output reg q  
);  
    always @(negedge clk) begin  
        q <= d;  
    end  
endmodule
```

# Flip-Flops

- Several D Flip-flops may be set together with a common clock to form a register.



## Exercise:

- 1- The operation of the flip flop is slow?
- a) True
  - b) False

## Answer:

1- The operation of the flip flop is slow?

- a) True
- b) False

# Blocking vs. Non-blocking assignment

# Blocking Assignment

- Verilog has two types of assignments that can take place inside always blocks with completely different behavior.
- Blocking assignment (**denoted by =**) completes before the next statement is executed.

```

module blocking (
    input a, b, c,
    output reg x, y
);
    always @(*) begin
        x = a & b;
        y = x | c;
    end
endmodule

```

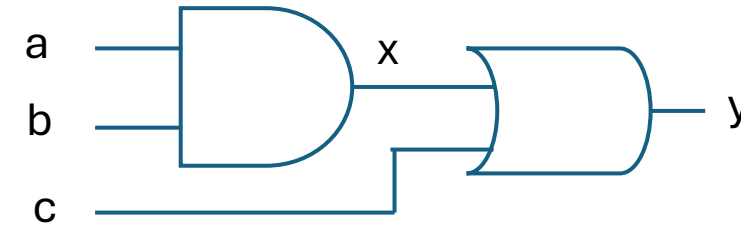
| Blocking behavior | a | b | c | x | y |
|-------------------|---|---|---|---|---|
| Initial state     | 1 | 1 | 0 | 1 | 1 |
| a changes         | 0 | 1 | 0 | 1 | 1 |
| x = a & b         | 0 | 1 | 0 | 0 | 1 |
| y = x   c         | 0 | 1 | 0 | 0 | 0 |

# Blocking Assignment

```

module blocking (
    input a, b, c,
    output reg x, y
);
    always @(*) begin
        x = a & b;
        y = x | c;
    end
endmodule

```



Multi-stage Combinational logic



# Non-Blocking Assignment

- Verilog has two types of assignments that can take place inside always blocks with completely different behavior.
- Non-blocking assignment (**denoted by <=**) assignments are deferred until all the RHS have been evaluated.

```

module non_blocking (
    input a, b, c,
    output reg x, y
);
    always @(*) begin
        x <= a & b;
        y <= x | c;
    end
endmodule

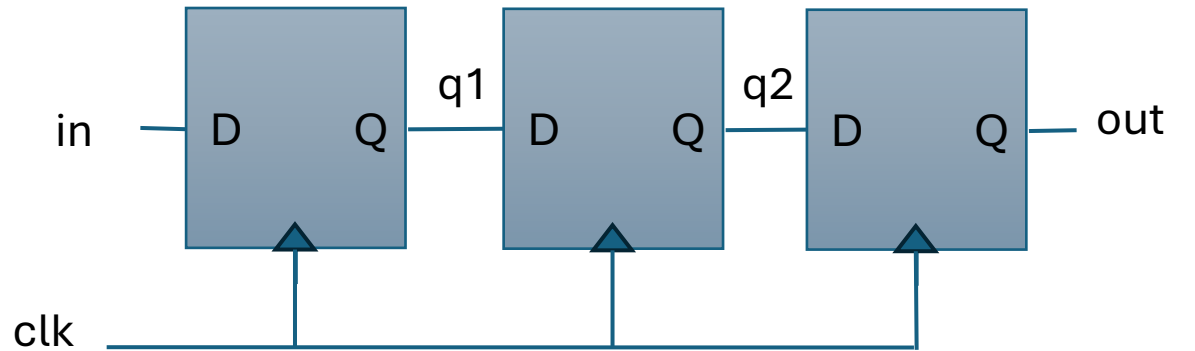
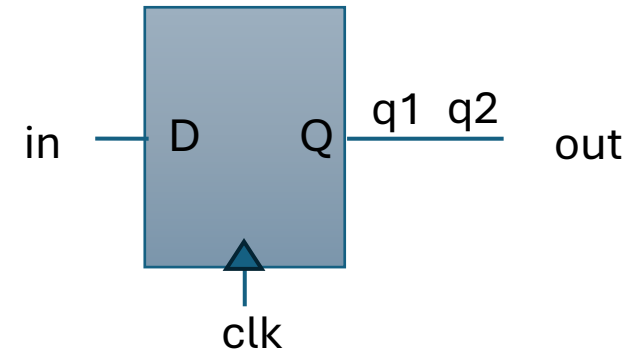
```

| Nonlocking behavior | a | b | c | x | y | Deferred     |
|---------------------|---|---|---|---|---|--------------|
| Initial state       | 1 | 1 | 0 | 1 | 1 |              |
| a changes           | 0 | 1 | 0 | 1 | 1 |              |
| x <= a & b          | 0 | 1 | 0 | 1 | 1 | X <= 0       |
| y <= x   c          | 0 | 1 | 0 | 1 | 1 | X<=0, Y <= 1 |
| assignment          | 0 | 1 | 0 | 0 | 1 |              |

# Non-Blocking Assignment

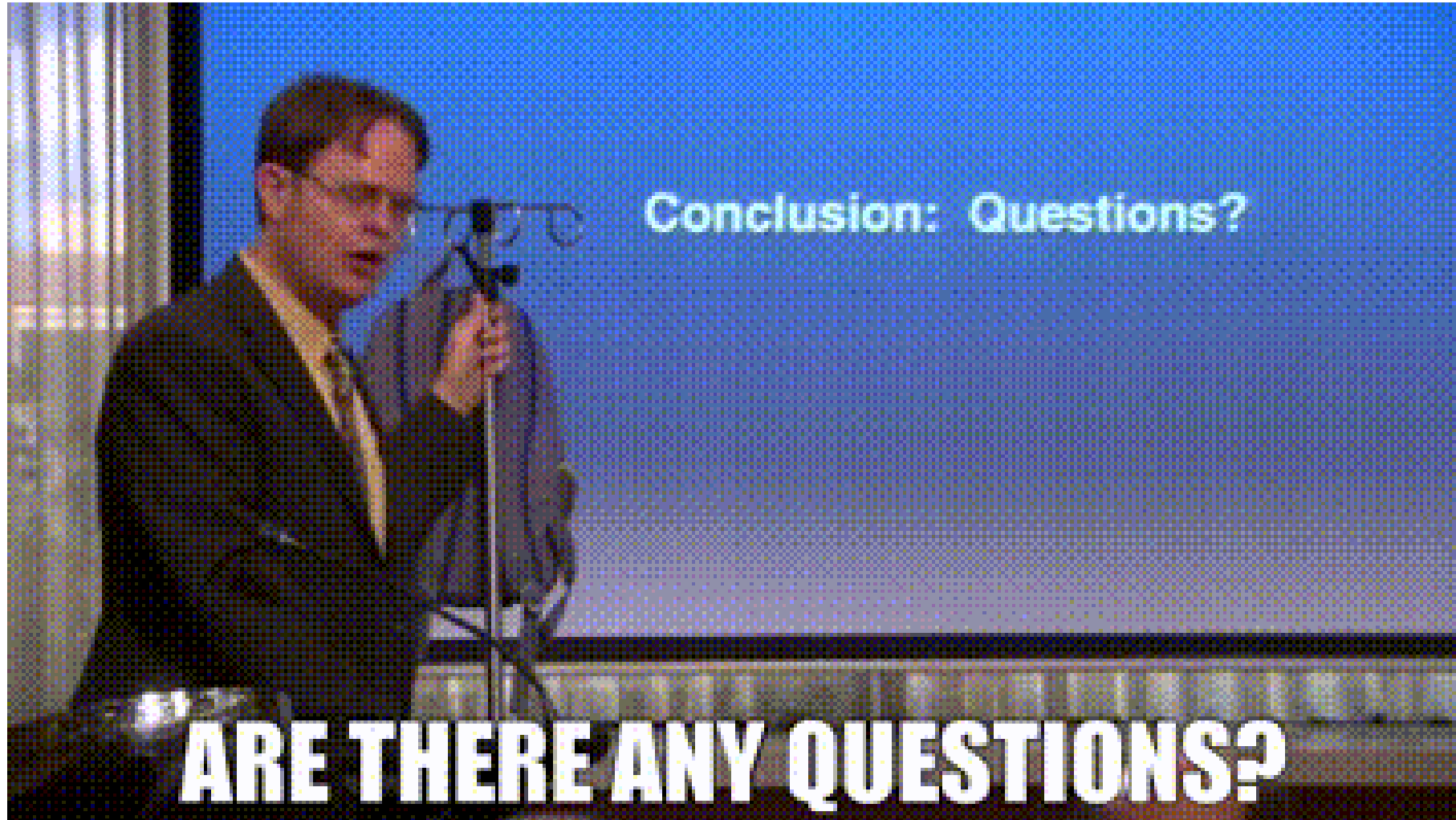
```
always @(posedge clk)
begin
    q1 = in;
    q2 = q1;
    out = q2;
end
```

```
always @(posedge clk)
begin
    q1 <= in;
    q2 <= q1;
    out <= q2;
end
```

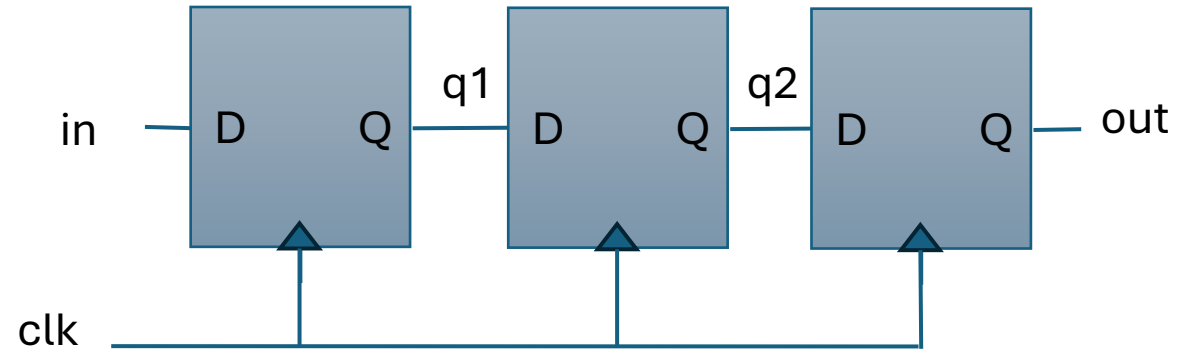


# Blocking vs. Non-Blocking

- Blocking assignments do NOT show the behavior of **multi-stage** sequential logic.
- **Guideline:** use Nonblocking assignments for sequential logic.
- Non-blocking assignments do NOT show the behavior of **multi-stage** combinational logic.
- **Guideline:** use blocking assignments for combinational logic.



1. How to implement the following circuit using blocking assignments?



# Answer:

1.

```
always @(posedge clk)
begin
    out = q2;
    q2 = q1;
    q1 = in;
end
```

# Synchronous vs. Asynchronous signals

# Sync vs. Async

- Sequential circuits must have control signals that return the system back to a “known state”.
- **Reset “Clear”** and **Set “Preset”** signals are commonly used to initialize the circuit into a defined state (**0** and **1**, respectively).
- For **synchronous** reset, reset takes effect only on the clock edge
- For **asynchronous** reset, reset takes effect as soon as it is asserted.



# Synchronous reset D-FF

```
module dff_async_rst (  
    input clk,  
    input rst_n,  
    input d,  
    output reg q  
);  
    always @(posedge clk) begin  
        if (!rst_n) begin                //Active low reset signal  
            q <= 0;  
        end else begin  
            q <= d;  
        end  
    end  
end  
endmodule
```

# Asynchronous reset D-FF

```
module dff_async_rst (  
    input clk,  
    input rst_n,  
    input d,  
    output reg q  
);  
    always @(posedge clk or negedge rst_n) begin  
        if (!rst_n) begin                                //Active Low reset signal  
            q <= 0;  
        end else begin  
            q <= d;  
        end  
    end  
end  
endmodule
```

# Lab2

Create 8 D flip-flops with active high asynchronous reset. All DFFs should be triggered by the positive edge of clk.

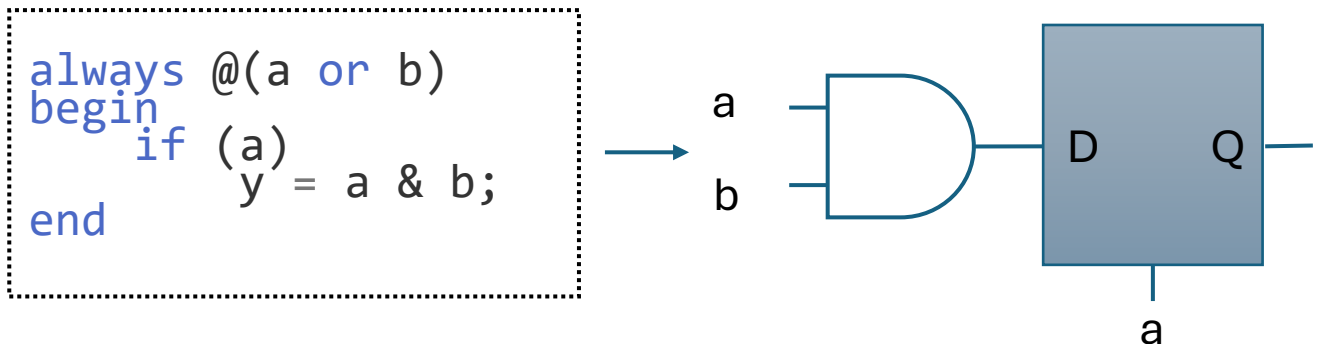
# Lab Solution

```
module dff_array (  
    input clk,  
    input rst,  
    input [7:0] d,  
    output reg [7:0] q  
);  
    always @(posedge clk or posedge rst) begin  
        if (rst) begin  
            q <= 8'h00;  
        end else begin  
            q <= d;  
        end  
    end  
end  
endmodule
```

# Inferred Latch

# Inferred Latch

- An **inferred** latch occurs when a combinational always block does not fully define all output cases.
- The synthesizer infers a memory element (latch) to **hold the previous value** when no conditions are met.
- This is usually **unintentional and should be avoided**, unless explicitly designing a latch.



# Inferred Latch – If Statement

- **Case 1: Missing else clause**

```
always @(*) begin
    if (a)
        y = b;
    else if (b)
        y = a;
    else if (c)
        y = d;
    else if (d)
        y = e;
    else if (e)
        y = c;
end
```

How to fix?



```
always @(a or b or c or d or e)
begin
    if (a)
        y = b;
    else if (b)
        y = a;
    else if (c)
        y = d;
    else if (d)
        y = e;
    else if (e)
        y = c;
    else
        y = 0; // Define a
                // default value to avoid latch
end
```



# Inferred Latch – If Statement

- **Case 2: Incomplete assignment**

```
always @(a or b or c)
begin
    if (a) begin
        y1 = b;
    end
    else if (b) begin
        y2 = a;
    end
    else begin
        y1 = 0;
        y2 = 0;
    end
end
```

How to fix?



```
always @(a or b or c) begin
    if (a) begin
        y1 = b;
        y2 = 0;
    end
    else if (b) begin
        y1 = 0;
        y2 = a;
    end
    else begin
        y1 = 0;
        y2 = 0;
    end
end
```

# Inferred Latch – If Statement

- **Case 2: Incomplete assignment**

```
always @(a or b or c)
begin
    if (a) begin
        y1 = b;
    end
    else if (b) begin
        y2 = a;
    end
    else begin
        y1 = 0;
        y2 = 0;
    end
end
```

How to fix?



```
always @(a or b or c) begin
    y1 = 0;
    y2 = 0;
    if (a) begin
        y1 = b;
    end
    else if (b) begin
        y2 = a;
    end
    else begin
        y1 = 0;
        y2 = 0;
    end
end
```

# Inferred Latch – Case Statement

- **Case 1: Missing default case**

```
always @(*) begin
  case (op_code)
    2'b00: y = a + b;
    2'b01: y = a - b;
    2'b10: y = a * b;
  endcase
end
```

How to fix?



```
always @(*) begin
  case (op_code)
    2'b00: y = a + b;
    2'b01: y = a - b;
    2'b10: y = a * b;
    default: y = 0;
  endcase
end
```

# Event Queue

# Inferred Latch – Case Statement

## • Case 2: Incomplete assignment

```
always @(*) begin
  case (sel)
    2'b00: begin
      y1 = a;
      y2 = b;
    end
    2'b01: begin
      y1 = b;
      y2 = a;
    end
    2'b10: begin
      y1 = c;
      y2 = d;
    end
    default: y1 = d;
  endcase
end
```

How to fix?

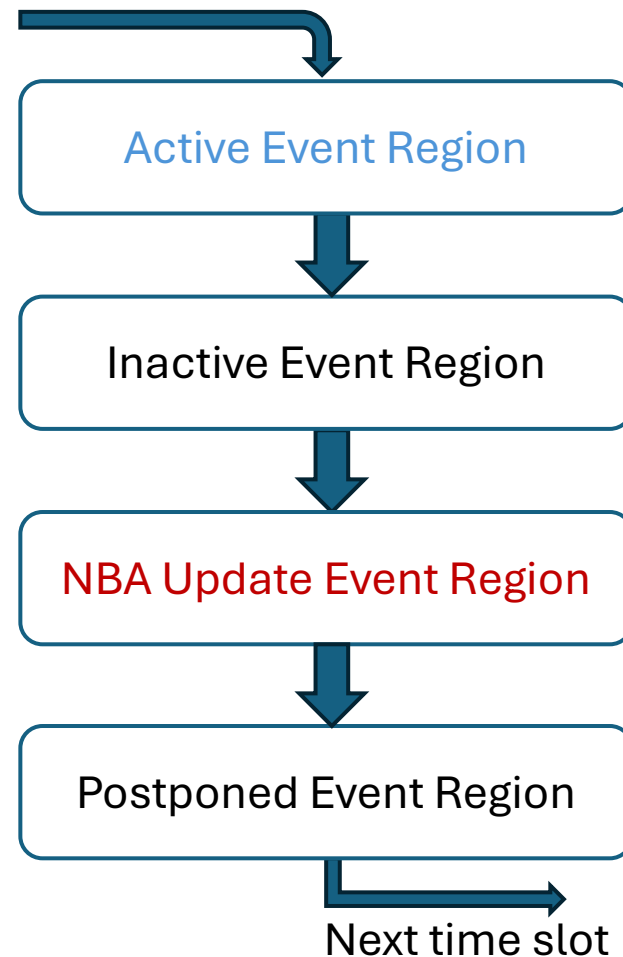


```
always @(*) begin
  case (sel)
    2'b00: begin
      y1 = a;
      y2 = b;
    end
    2'b01: begin
      y1 = b;
      y2 = a;
    end
    2'b10: begin
      y1 = c;
      y2 = d;
    end
    default: begin
      y1 = d;
      y2 = 0;
    end
  endcase
end
```



# Event Queue

Previous time slot



- Programming statements
- Evaluate & update blocking assignments
- Evaluate nonblocking assignments
- Continuous assignments
- \$display

- #0 delayed events

```
initial begin
    #0 a1 = 1;
    a2 = 2;
end
```

- Update LHS of nonblocking assignment

```
initial begin
    a1 <= 1;
    a2 = 2;
end
```

- \$strobe
- \$monitor

## Exercise:

What will be the displayed values of x, y and z?

```
integer x, y, z;  
initial begin  
    x = 10;  
    y = 20;  
    z = 30;  
  
    $display("%d, %d, %d", x, y, z);  
  
    x <= 40;  
    y <= 50;  
    z <= 60;  
  
    $display("%d, %d, %d", x, y, z);  
  
    x = 70;  
    y = 80;  
    z = 90;  
  
    $strobe("%d, %d, %d", x, y, z);  
    $display("%d, %d, %d", x, y, z);  
end
```

# Answer:

```
integer x, y, z;  
initial begin  
    x = 10;  
    y = 20;  
    z = 30;  
  
    $display("%d, %d, %d", x, y, z);           // 10, 20, 30  
  
    x <= 40;  
    y <= 50;  
    z <= 60;  
  
    $display("%d, %d, %d", x, y, z);           // 10, 20, 30  
  
    x = 70;  
    y = 80;  
    z = 90;  
  
    $strobe("%d, %d, %d", x, y, z);             // 40, 50, 60  
    $display("%d, %d, %d", x, y, z);           // 70, 80, 90  
end
```



# Race Condition

- A race condition occurs when the behavior of a system depends on the order or timing of events that are not guaranteed, leading to non-deterministic results.

# Race Condition

- **Case 1: Read-Write Race**

```
assign a = b & c;  
always @ (b or d)begin  
    if(a)  
        o = b & d;  
    else  
        o = b | d;  
end
```

How to fix?



```
assign a = b & c;  
always @ (*)begin  
    if(a)  
        o = b & d;  
    else  
        o = b | d;  
end
```

# Race Condition

- **Case 2: Write-Write Race**

```
always@(b or c)begin
    if(b != c)
        flag = 1;
    else
        flag = 0;
end

always@(b or d)begin
    if(b == d)
        flag = 1;
    else
        flag = 0;
end
```

How to fix?



```
always@(b or c or d)begin
    if(b == d || b != c)
        flag=1;
    else
        flag=0;
end
```



# Coding Guidelines

1. When modeling sequential logic, use nonblocking assignments.
2. When modeling combinational logic, use blocking assignments.
3. When modeling both sequential and combinational logic within the same always block, use nonblocking assignments.
4. Don't mix blocking and nonblocking assignments in the same always block.
5. **DON'T** make assignments to the same variable from more than one always block.

